CPEN 411 Assignment 2: High-Performance Cache Policies for Last Level Cache (LLC)

Sidharth Sudhir (11255635), Harnoor Saigal (28703262)

Implementation Strategy

1. Random

The Random policy does not need initialization or an update state. Our implementation invokes a Mersenne Twister random number generator with a uniform distribution over the set's associativity when selecting a victim.

2. Least Frequently Used (LFU)

LFU tracks reuse frequency and evicts the line with the lowest count. The design constraints for this policy are as follows:

- 4-bit saturating counter (0-15) per cacheline.
- Random victim chosen if multiple ways share the same frequency.
- If all ways have the same non-zero frequency during eviction, reset all counters to zero.

We implemented the LFU policy as follows:

- Use `std::vector<uint8_t> frequency_counter` sized `NUM_SET × NUM_WAY`.
- Increment the counter on hit (capped at 15) and reset inserted line's counter to 0 on miss.
- To find the victim, we first find the counter with the lowest frequency. If all the counters are equal and non-zero, we reset all the counters to 0. If we have multiple candidates with the same frequency, we break ties by randomly selecting a victim out of the candidates.

The following assumptions were made:

- When there is a cache miss, reset the counter to 0.
- If all the counters have the same non-zero value during eviction, they are reset to 0 and a victim is picked randomly from the whole set because we would have `NUM_WAY` counters at 0.

3. Psuedo-Least Recently Used (pLRU)

pLRU mimics accurate LRU functionality while reducing overhead bits. It uses a binary tree structure, where each node records the direction of the least recently used path. We implemented the pLRU policy as follows:

- Each set uses `std::vector<uint8_t> plru_bits` to store `(NUM_WAY - 1)` bits in heap order (binary tree layout).
- To find a victim, we start from the root and follow the bit values (0 -> left, 1 -> right) until reaching a leaf. The corresponding way is chosen as the victim.

- On an access, we traverse the path from the root to the accessed line and flip the bits along the path to point away from the accessed subtree. This marks the accessed line as MRU and designates the opposite side as LRU.

4. Bimodal Insertion Policy (BIP)

BIP inserts most new lines at the LRU position and occasionally inserting at the MRU position with a small probability (ϵ). This allows one-time-use lines to be quickly evicted while protecting potentially hot lines. The design constraints for this policy is that ϵ is fixed at 5% (1 in 20 insertions should go to MRU). We implemented the BIP policy as follows:

- Each line maintains a timestamp using `std::vector<uint64_t> last_used_cycles.`
- A global `bimodal_counter` is incremented on each miss.
- On a hit, the line is promoted to MRU by setting its timestamp to `current_cycle`.
- On a miss:
    - With probability ϵ (enforced when `bimodal_counter % 20 == 0`), the line is inserted at MRU (`timestamp = current_cycle`).
    - Otherwise, the line is inserted at LRU by assigning it a timestamp one less than the current minimum in the set.
- Victim lines are always selected using the minimum timestamp.

5. Dynamic Insertion Policy (DIP)

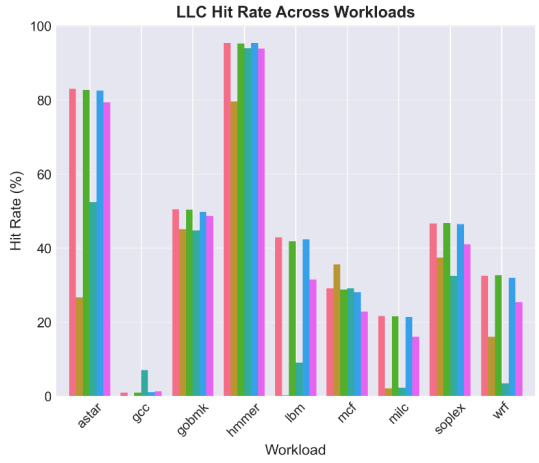
DIP adapts between LRU and BIP using set dueling, guided by leader sets and a global selector counter (PSEL). The design constraints for this policy are as follows:

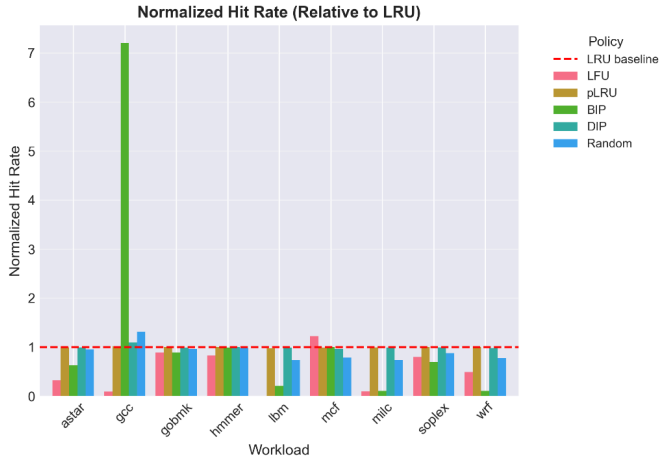
- 64 leader sets per policy (64 for LRU, 64 for BIP).
- PSEL is a 10-bit saturating counter, initialized at 512.
- BIP uses ϵ value of 5%.

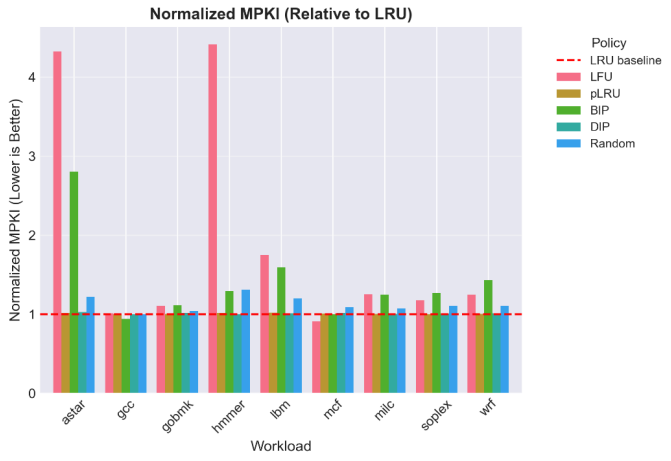
We implemented the DIP policy as follows:

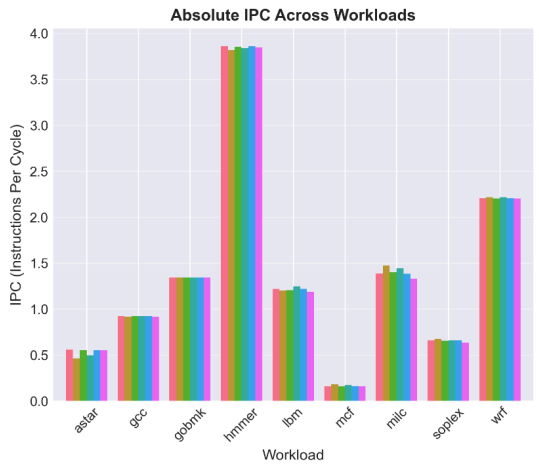
- Each line maintains its timestamp using `std::vector<uint64_t> last_used_cycles.`
- The function `identify_set_type()` partitions sets into constituencies and classifies them as LRU leaders, BIP leaders, or followers.
- On a hit, the accessed line is promoted to MRU using LRU insertion.
- On a miss:
    - In LRU leader sets, apply LRU insertion and increment PSEL if it is less than 1023.
    - In BIP leader sets, apply BIP insertion (ϵ = 5%) and decrement PSEL if it is greater than 0.
    - In follower sets, insertion depends on PSEL: if PSEL ≥ 512, apply BIP insertion; otherwise, apply LRU insertion.
- Victim lines are always selected using the minimum timestamp.

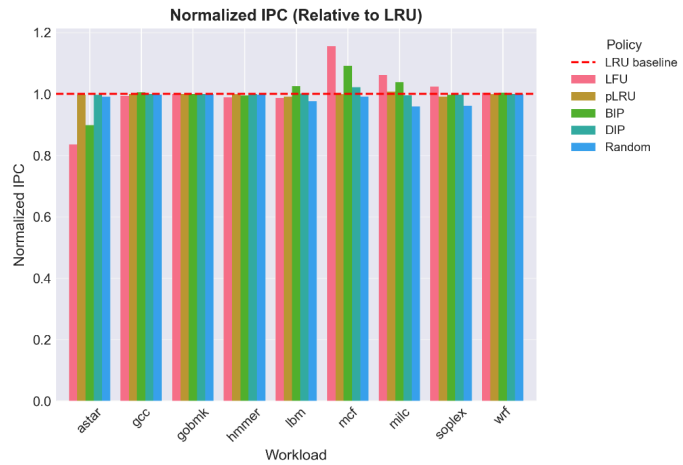
Performance Analysis of Policies Under Different Workloads

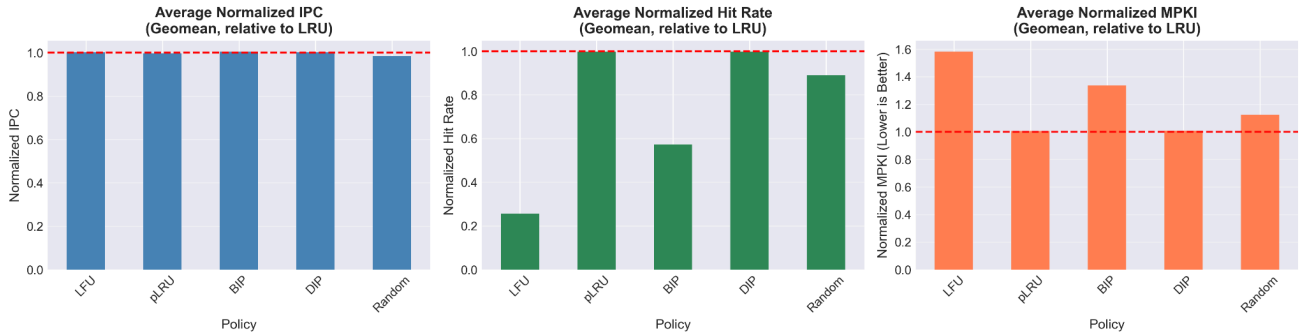












Result Findings

We evaluated 5 different policies against 3 different metrics (i.e. IPC, Hit Rate, MPKI). The table below summarizes the overall performance based on the graphs in page 4:

Policies	**Performance Metrics (w.r.t LRU Performance)**		
	IPC	**MPKI**	**Hit Rate**
Random	-1.40%	-12.19%	-10.98%
pLRU	-0.16%	+0.56%	-0.22%
LFU	+0.26%	-58.18%	-74.44%
BIP	+0.50%	-33.69%	-42.75%
DIP	+0.19%	+0.85%	-0.21%

Following are some of the findings from the output data:

- For workloads with strong locality patterns (e.g., astar, gobmk, hmmer, wrf), pLRU along with DIP exhibited performance metrics nearly on par with traditional LRU, differing by minimal IPC margins (below 0.05%). This observation aligns with expectations, as these workloads depend on the recency of cacheline accesses.
- In workloads that are prone to thrashing, such as lbm and mcf, LFU often underperforms. This could be because its 4-bit counters are quickly saturated, making it harder to distinguish between truly frequent and less useful lines, making it a less effective choice of policy.
- In gcc, all policies delivered very similar results because the high miss rate left little room for optimization. In milc, however, policies that favored recency (like LRU and pLRU) maintained an advantage, while alternatives such as LFU and BIP lagged behind.
- pLRU delivers nearly equivalent performance to conventional LRU at a lower hardware cost, making it a power and resource effective option over traditional LRU.

- The DIP policy consistently delivered stable results where it avoided the weaknesses of LRU in thrash-heavy workloads while still staying very close to LRU's performance in more locality-friendly phases.
- BIP's strength is clear in workloads with thrashing, but it can actually hurt performance in workloads where high hit rates are critical.
- LFU showed mixed results where it can work well in workloads with stable reuse patterns, but it often underperforms when access patterns change quickly.
- The Random policy performed the weakest overall, since it ignores all locality and reuse information.